

2-D arrays in Java

Scenario-Based Questions for 2-D Array in Java

1. Student Marks Management — Basic ☐

A teacher wants to store the marks obtained by **5 students in 3 subjects**: Java, DBMS, and Mathematics.

Use a 2-D array where:

- Each row represents a student.
- Each column represents a subject.

Task:

Write a Java program to:

1. Store the marks of 5 students.
2. Display the marks in tabular form.
3. Calculate the total marks obtained by each student.
4. Calculate the average marks of each student.
5. Find the student who obtained the highest total marks.

Example:

Student	Java	DBMS	Maths
Student 1	78	85	90
Student 2	65	72	80
Student 3	88	91	84
Student 4	70	68	75
Student 5	92	89	95

2. Classroom Attendance System ☐

A college wants to maintain the attendance of **5 students for 7 days**.

Use:

- 1 → Present
- 0 → Absent

Task:

Write a Java program using a 2-D array to:

1. Store attendance data.

2. Display the attendance of all students.
3. Calculate the number of days each student was present.
4. Calculate attendance percentage.
5. Display students whose attendance is below 75%.

Example:

Student	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Student 1	1	1	0	1	1	1	0
Student 2	1	0	1	1	1	0	1

3. Cinema Seat Booking System ☐ ☐

A cinema hall contains **5 rows and 6 seats in each row**.

Use a 2-D array to represent the seats:

0 → Available
1 → Booked

Task:

Write a Java program to:

1. Display the seating arrangement.
2. Allow a user to select a row and seat number.
3. Check whether the seat is available.
4. Book the seat if available.
5. Display the updated seating arrangement.
6. Count the total available seats.

Example:

```
0 0 1 0 1 0
1 0 0 0 1 1
0 0 0 1 0 0
```

4. Sales Analysis of a Company ☐ ☐

A company has **4 branches** and records sales for **3 months**.

Rows represent branches and columns represent months.

	Jan	Feb	Mar
Branch 1	120	150	180
Branch 2	200	170	210
Branch 3	140	160	155

Branch 4	250	220	280
----------	-----	-----	-----

Task:

Write a Java program to:

1. Store the sales data in a 2-D array.
 2. Calculate total sales of each branch.
 3. Calculate total sales for each month.
 4. Find the branch with the highest sales.
 5. Find the month with the highest overall sales.
-

5. Matrix Addition ☐ ☐

A software application needs to add two matrices.

Given:

Matrix A

1	2	3
4	5	6
7	8	9

and

Matrix B

9	8	7
6	5	4
3	2	1

Task:

Write a Java program to:

1. Create two 2-D arrays.
2. Read the elements from the user.
3. Add corresponding elements.
4. Store the result in a third 2-D array.
5. Display the resultant matrix.

Expected result:

10	10	10
10	10	10
10	10	10

6. Find Highest Marks in Each Subject □ □

A university stores marks of **6 students in 4 subjects** using a 2-D array.

Task:

Write a Java program to determine:

- Highest marks in each subject.
- Student who obtained the highest marks in each subject.
- Average marks of each subject.

This problem helps students understand **column-wise processing** of a 2-D array.

7. Hospital Patient Temperature Monitoring □ □ □

A hospital records the temperature of **5 patients for 4 consecutive days**.

The data is stored in a 2-D array.

Task:

Write a Java program to:

1. Store temperature readings.
 2. Display all readings.
 3. Find the average temperature of each patient.
 4. Find the highest temperature recorded.
 5. Identify the patient and day on which the highest temperature was recorded.
 6. Count the number of readings above 38°C.
-

8. Parking Lot Management System □ □ □

A shopping mall has a parking area containing **4 rows and 5 parking slots**.

Use:

0 → Empty
1 → Occupied

Task:

Develop a Java program that:

1. Displays the parking layout.

2. Counts occupied spaces.
 3. Counts available spaces.
 4. Allows a vehicle to occupy an empty slot.
 5. Allows a vehicle to leave a parking slot.
 6. Displays the updated parking layout.
-

9. Monthly Electricity Consumption ☐ ☐ ☐

An electricity company maintains the electricity consumption of **5 households for 6 months**.

Task:

Using a 2-D array:

1. Store monthly consumption.
 2. Calculate total consumption for each household.
 3. Calculate average monthly consumption.
 4. Find the household with maximum consumption.
 5. Find the month with the highest overall consumption.
 6. Identify households whose average consumption exceeds a specified limit.
-

10. Image Pixel Processing — Advanced ☐ ☐ ☐ ☐

A grayscale image is represented as a 2-D array where each element represents the intensity of a pixel between 0 and 255.

Example:

10	20	30
40	50	60
70	80	90

Task:

Write a Java program to:

1. Store the image pixels in a 2-D array.
2. Display the image matrix.
3. Find the brightest pixel.
4. Find the darkest pixel.
5. Calculate the average pixel intensity.
6. Convert the image into a binary image using:
 - $\text{Pixel} \geq 128 \rightarrow 1$
 - $\text{Pixel} < 128 \rightarrow 0$

One Good Problem for a Java Lab

If you want **one comprehensive scenario** that covers most 2-D array concepts, I recommend this:

□ University Examination Result System

A university wants to develop a simple examination result system for a class of **N students and M subjects**. The marks of every student are stored in a 2-D array.

Write a Java program that:

1. Accepts the number of students and subjects.
2. Stores marks using a 2-D array.
3. Displays the marks in tabular form.
4. Calculates the **total marks of every student**.
5. Calculates the **average marks of every student**.
6. Finds the **highest marks in each subject**.
7. Finds the **average marks in each subject**.
8. Identifies the **student with the highest total marks**.
9. Counts the number of students who have failed in at least one subject.
10. Displays students whose overall average is greater than 75%.

Concepts covered

This single problem allows you to teach:

- 2-D array declaration
- Array initialization
- Taking input using `Scanner`
- Nested `for` loops
- Row-wise traversal
- Column-wise traversal
- `array.length`
- `array[i].length`
- Searching in a 2-D array
- Finding maximum/minimum
- Sum and average
- Conditional processing
- Passing a 2-D array to a method

A useful teaching progression is **Marks → Attendance → Matrix Operations → Seat/Parking System → Image Processing**, because students gradually move from simple row/column traversal to real-world applications.